

תרגיל בית 3

פונקציות ומטריצות

הנחיות כלליות:

• קראו היטב את השאלות והקפידו שהתכניות שלכם פועלות בהתאם לנדרש.

• את התרגיל יש לפתור לבד!

• הקפידו על כללי ההגשה המפורסמים באתר. בפרט, יש להגיש את כל השאלות יחד בקובץ `ex1_012345678.py` המצורף לתרגיל, לאחר החלפת הספרות 012345678 במספר ת.ז שלכם, כל 9 הספרות כולל ספרת הביקורת.

• מועד אחרון להגשה: כמפורסם באתר.

• בדיקה עצמית: כדי לוודא את נכונותן ואת עמידותן של התוכניות לקלטים שגויים, בכל שאלה, הריצו את תוכניתכם עם מגוון קלטים שונים, אלה שהופיעו כדוגמאות בתרגיל וקלטים נוספים עליהם חשבתם (וודאו כי הפלט נכון וכי התוכנית אינה קורסת).

• היות ובדיקת התרגילים עשויה להיות אוטומטית, יש להקפיד על פלטים מדויקים על פי הדוגמאות (עד לרמת הרווח)

• אופן ביצוע התרגיל: בתרגיל זה עליכם להשלים את הקוד בקובץ המצורף.

• אין לשנות את שמות המשתנים שכבר מופיעים בקובץ השלד של התרגיל. יש לעבוד עם המשתנים שמופיעים בשלד התרגיל. על הקוד של כל שאלה לעבוד ולספק את התוצאה הדרושה עבור קלט שיוזן במשתנים שמופיעים בשלד (יש להחליף את סימני השאלה שמופיעים ליד שמות משתני הקלט בערכים לבחירתכם ולהגיש את התרגיל ללא סימני השאלה). יחד עם זאת, אתם רשאים להוסיף משתנים נוספים כראות עינכם.

• אין למחוק את ההערות שמופיעות בשלד.

שאלה 1:

כתבו את הפונקציה `is_in_range(lst1, a, b)` המקבלת רשימת מספרים `lst1`, ושני מספרים ממשיים `a` ו-`b`. הפונקציה תחזיר `True` אם כל האיברים ברשימה גדולים ממש `a` וגם קטנים ממש `b`, ותחזיר `False` אחרת.

- במקרה ו-`lst1` היא רשימה ריקה יש להחזיר `True`.
- ניתן להניח כי `a < b`

דוגמאות הרצה:

```
>>> is_in_range([1,3,5,2,3,7],0,10)
True
>>> is_in_range([1,3,5,2,3,7],1,10)
False
>>> is_in_range([1,3,5,2,3,7],0,7)
False
```

שאלה 2:

כתבו פונקציה `first_upper_str(lst2)` המקבלת רשימה של אובייקטים כלשהם (מטיפוס כלשהו) **ומחזירה** (לא מדפיסה למסך) את **האינדקס** (לא את האיבר ברשימה) של המחרוזת הראשונה ברשימה שכל האותיות בה הינן אותיות גדולות. במידה ולא קיים איבר כזה, יש להחזיר את הערך `-1`, כמו כן, עבור רשימה ריקה יש להחזיר `-1` גם כן.

```
>>> first_upper_str([1, 2, 3, 55, 'FF', 11, False])
4
>>> first_upper_str([])
-1
>>> first_upper_str([1, 2, 3, 55, 'FF ', 11, False])
4
>>> first_upper_str([1, 2, 3, 55, 'aBCD', 11, False])
-1
```

שאלה 3:

א. ממשו את הפונקציה ***drop_duplicates(lst3)*** המקבלת רשימה של מספרים בשם `lst3` ומחזירה רשימה חדשה הכוללת את כל איברי הרשימה `lst3` כאשר הם מופיעים לפי סדר הופעתם המקורי אך ללא כפילויות.

שימו לב:

- סדר המספרים ברשימת הפלט יקבע בהתאם למיקום המופע הראשון של אותו מספר ברשימה.

- אין לשנות את הרשימה המקורית (`lst3`)

ב. ממשו את הפונקציה ***drop_duplicates_in_place(lst3)*** המקבלת רשימה של מספרים בשם `lst3` ומשנה את רשימת הקלט `lst3` כך שלא תכיל כפילויות, תוך שמירה על סדר הופעת האיברים בה.

שימו לב:

- פונקציה זו תשנה את רשימת הקלט `lst3` מבלי ליצור רשימה חדשה.
- פונקציה זו לא תחזיר אף ערך.

דוגמאות הרצה:

```
>>> lst3 = [1,2,3,2,4]
>>> drop_duplicates(lst3)
[1, 2, 3, 4]
>>> print(lst3)
[1, 2, 3, 2, 4]
>>> drop_duplicates_in_place(lst3)
>>> print(lst3)
[1, 2, 3, 4]
```

שאלה 4:

ממשו את הפונקציה *is_contain(lst4, lst5)* המקבלת שתי רשימות lst4 ו-lst5. יש להחזיר **רשימה חדשה** אשר תהיה באורך הרשימה lst5. עבור כל איבר ב-lst5 יש לבדוק אם הוא נמצא ברשימה lst4 ואם כן - ברשימה החדשה יופיע (באותו המיקום בו מופיע האיבר המקורי ברשימה lst5) **האינדקס** של אותו איבר ברשימה lst4. אם האיבר מופיע כמה פעמים ברשימה lst4 יש לקחת את האינדקס של המופע הראשון, אם האיבר לא מופיע ב-lst4 יופיע ברשימה החדשה -1.

הערה : הרשימות יכולות להיות ריקות

דוגמאות הרצה :

```
>>> lst4 = [1,2,4,2,7,9,12]
>>> lst5 = [0,2,4,11,2,21,12,13]
>>> is_contain(lst4, lst5)
[-1, 1, 2, -1, 1, -1, 6, -1]
>>> lst4 = []
>>> lst5 = [0,2,4,11,2,21,12,13]
>>> is_contain(lst4, lst5)
[-1, -1, -1, -1, -1, -1, -1, -1]
>>> lst4 = [1,2,5,2,31,2,4,6]
>>> lst5 = []
>>> is_contain(lst4, lst5)
[]
```

מטריצות:

בשאלות הבאות נעבוד עם מטריצות של מספרים שלמים. המטריצות ייוצגו בצורה הבאה :
 מטריצה שמימדיה m על n (כלומר, יש לה m שורות שאורך כל אחת מהן n) תיוצג על ידי רשימה של
 רשימות, שכל אחת מהן מייצגת שורה אחת.
 כך לדוגמא, המטריצה שבעלת 3 שורות ו 2 עמודות שערכיה הם :

$$\begin{pmatrix} 2 & 1 \\ 4 & 3 \\ 6 & 5 \end{pmatrix}$$

תיוצג על ידי הרשימה

[[2,1], [4,3], [6,5]]

ניתן להניח שהמטריצות בשאלות הבאות אינן ריקות.

שאלה 5:

ממשו פונקציה בשם `mul_elementwise(mat1, mat2)` המקבלת 2 מטריצות בשם `mat1` ו-`mat2` תוך
 שימוש בייצוג שהוצג לעיל, ומחזירה מטריצה חדשה שהיא תוצאת הכפלת איבר-איבר של שתי
 המטריצות. כלומר : `new_mat[i][j] = mat1[i][j] * mat2[i][j]`

במידה והממדים של שתי המטריצות אינם תואמים, יש להדפיס את ההודעה הבאה :

Can't multiply elementwise mat with shape (<M1> , <N1>) and mat with shape
 (<M2> , <N2>)

כאשר `M1` מייצג את המספר השורות של `mat1` ו-`N1` את מספר העמודות של `mat1` (ובאופן זהה עבור
`mat2`).

הערות :

- אין לשנות את `mat1` ואת `mat2`.
- ניתן להניח כי המטריצות תקינות (מכילות יותר לפחות שורה אחת, וכל השורות באותו האורך)
- אין להניח כי המטריצות הינן בממדים זהים (יש לבדוק זאת ולהדפיס הודעה בהתאם, כפי שצוין
 (מעלה)

דוגמאות הרצה:

```
>>> mat1 = [[1,2,3], [4,5,6], [7,8,9]]
>>> mat2 = [[10,100,1000], [20,30,40], [-10,-100,-1000]]
>>> mul_elementwise(mat1, mat2)
[[10, 200, 3000], [80, 150, 240], [-70, -800, -9000]]
>>> mat1 = [[1,2,3], [4,5,6], [7,8,9]]
>>> mat2 = [[1,1,1], [2,2,2]]
>>> mul_elementwise(mat1, mat2)
Can't multiply elementwise mat with shape (3, 3) and mat with shape (2, 3)
>>> mat1 = [[1,2,3], [4,5,6], [7,8,9]]
>>> mat2 = [[10,100], [20,30], [-10, -100]]
>>> mul_elementwise(mat1, mat2)
Can't multiply elementwise mat with shape (3, 3) and mat with shape (3, 2)
```

שאלה 6:

ממשו את הפונקציה *mat_mul_vec(mat3, vec)* המקבלת מטריצה תקנית mat3 ווקטור (מיוצג ע"י רשימה בודדת בשם vec המכילה מספרים מסוג int/float) ומחזירה וקטור חדש שהוא תוצאה של הכפלת המטריצה בוקטור (הכפלה מימין של הוקטור בוקטור עמודה).

תזכורת:

$$\text{output}[i] = \sum_{j=0}^{N-1} \text{mat1}[i][j] * \text{vec}[j]$$

אם ממדי הוקטור והמטריצה אינם תואמים (מספר העמודות של המטריצה צריך להיות שווה לאורך הוקטור) יש להדפיס את ההודעה הבאה:

```
Can't multiply mat with shape (<M1> , <N1>) and a vector with length
<len(vec)>
```

דוגמאות הרצה :

```
>>> mat3 = [[1,1],[2,2],[3,3],[4,4]]
```

```
>>> vec = [10,100]
```

```
>>> mat_mul_vec(mat3, vec)
```

```
[110, 220, 330, 440]
```

```
>>> mat3 = [[1,1],[2,2],[3,3],[4,4]]
```

```
>>> vec = [10, 100, 1000, 10000]
```

```
>>> mat_mul_vec(mat3, vec)
```

Can't multiply mat with shape (4, 2) and a vector with length 4

בהצלחה!